

Informe: Simulador de Sistemas de Archivos

FAT y Ext2

Análisis y Comparación de Mecanismos de Gestión de Bloques

Angel Besteiro
Brigido Noguera

9 de octubre de 2025

Índice

1. El Problema	2
2. Técnicas Computacionales para su Solución	2
2.1. Arquitectura General del Simulador	2
2.2. Estructuras de Datos Clave	3
2.2.1. Sistema FAT	3
2.2.2. Sistema Ext2 (Simplificado)	3
2.3. Algoritmos Principales	3
3. Análisis y Comparación de Resultados	4
3.1. Sistema FAT	4
3.2. Sistema Ext2	4
3.3. Comparación Directa	5
4. Conclusiones	5

1. El Problema

El objetivo fundamental de este proyecto es el diseño y la implementación de un simulador de sistemas de archivos en C++. Este simulador modela el comportamiento de dos arquitecturas clásicas de gestión de almacenamiento en disco a nivel de bloques: **FAT (File Allocation Table)** y una versión simplificada de **Ext2 (Second Extended Filesystem)**.

El propósito principal no es crear un sistema de archivos funcional para uso real, sino construir una herramienta didáctica que permita visualizar y comparar cómo cada sistema gestiona las operaciones básicas:

- **Creación de archivos:** Asignación de bloques de datos en el disco y registro de la estructura del archivo.
- **Lectura de archivos:** Localización y recuperación del contenido a partir de los metadatos del sistema.
- **Eliminación de archivos:** Liberación de los bloques de datos y eliminación de las referencias al archivo.

A través de la simulación, se busca analizar los efectos de la fragmentación, la eficiencia en el uso del espacio y la complejidad de las estructuras de datos subyacentes en cada uno de estos modelos.

2. Técnicas Computacionales para su Solución

La solución se desarrolló en C++ (C++17) utilizando un paradigma de programación orientada a objetos para encapsular la lógica de cada componente del sistema.

2.1. Arquitectura General del Simulador

La arquitectura se fundamenta en la separación de responsabilidades, con una clase abstracta `FileSystem` que define una interfaz común y clases derivadas que implementan la lógica específica de FAT y Ext2.

- **Clase Disk:** Simula el disco físico como un conjunto de bloques. Es un recurso compartido (gestionado mediante `std::shared_ptr`) sobre el cual operan los sistemas de archivos.
- **Clase Base Abstracta FileSystem:** Define el contrato que cualquier sistema de archivos debe cumplir, con métodos virtuales puros como `create()`, `read()` y `del()`. Esto permite tratar a ambos sistemas de forma polimórfica.
- **Clases Concretas FATFileSystem y Ext2FileSystem:** Heredan de `FileSystem` y sobreescriben los métodos para implementar la lógica de asignación y gestión de bloques propia de cada sistema.

2.2. Estructuras de Datos Clave

2.2.1. Sistema FAT

El sistema FAT se basa en una estructura de lista enlazada donde la "tabla" es el mapa que define los enlaces.

- **Tabla de Asignación de Archivos (FAT):** Implementada como un `std::vector<int>fat_table`. El índice del vector corresponde al número de bloque en el disco. El valor en esa posición indica el siguiente bloque del archivo. Se utilizan valores especiales como `FAT_EOF` (-1) para el fin de archivo y `FAT_FREE` (0) para un bloque libre.
- **Directorio:** Un `std::map<std::string, int>` que asocia el nombre de un archivo con su bloque de inicio. A partir de este bloque, se recorre la `fat_table` para encontrar el resto del contenido.

2.2.2. Sistema Ext2 (Simplificado)

El sistema Ext2 utiliza una asignación indexada, centralizando los metadatos de un archivo en una estructura llamada inodo.

- **Tabla de Inodos:** Implementada como un `std::vector<Inode>inode_table`. Cada archivo en el sistema tiene un inodo asignado.
- **Estructura Inode:** Contiene los metadatos del archivo. En nuestra simulación, incluye:
 - Tamaño del archivo (`size`).
 - Punteros directos (`std::vector<int>direct_pointers`): Un pequeño arreglo de punteros que apuntan directamente a los primeros bloques de datos del archivo.
 - Puntero indirecto simple (`int single_indirect`): Un puntero que apunta a un bloque que, en lugar de datos, contiene una lista de punteros a más bloques de datos. Esto permite manejar archivos de mayor tamaño.
- **Directorio:** Al igual que en FAT, se usa un `std::map<std::string, int>`, pero en este caso asocia el nombre del archivo con su **número de inodo**, no con un bloque.

2.3. Algoritmos Principales

- **Creación de Archivo (FAT):**
 1. Se calcula el número de bloques necesarios.
 2. Se buscan bloques libres en el disco.
 3. Se enlazan estos bloques en la `fat_table`, formando una lista enlazada. El último bloque apunta a `FAT_EOF`.
 4. Se escribe el contenido en los bloques del disco.
 5. Se añade la entrada (nombre, bloque inicial) al directorio.
- **Creación de Archivo (Ext2):**
 1. Se busca un inodo libre en la `inode_table`.

2. Se buscan los bloques de datos necesarios en el disco.
 3. Se asignan los bloques: primero se llenan los punteros directos del inodo.
 4. Si se necesitan más bloques, se asigna un bloque adicional para actuar como bloque de punteros indirectos, y se llena con las direcciones de los bloques de datos restantes.
 5. Se escribe el contenido en los bloques de datos.
 6. Se añade la entrada (nombre, número de inodo) al directorio.
- **Eliminación de Archivo (FAT):**
 1. Se localiza el bloque inicial a través del directorio.
 2. Se recorre la cadena de bloques en la `fat_table`.
 3. En cada paso, el bloque de disco se marca como libre y la entrada correspondiente en la `fat_table` se establece en `FAT_FREE`.
 4. Se elimina la entrada del directorio.

3. Análisis y Comparación de Resultados

Aunque el simulador no mide el tiempo de ejecución, permite analizar lógicamente el rendimiento y la eficiencia de cada sistema.

3.1. Sistema FAT

- **Simplicidad:** Su mayor ventaja es la facilidad de implementación. La lógica de seguir una lista enlazada es directa.
- **Fragmentación Externa:** Es su principal debilidad. Cuando los archivos se crean y eliminan repetidamente, los bloques de un nuevo archivo grande pueden quedar dispersos por todo el disco. Esto, en un disco mecánico real, se traduciría en un rendimiento de lectura muy bajo, ya que el cabezal del disco tendría que moverse constantemente.
- **Rendimiento de Lectura:** El acceso a los datos es secuencial por naturaleza. Para leer el bloque N de un archivo, es necesario recorrer los N-1 enlaces previos en la tabla FAT, lo que lo hace ineficiente para el acceso aleatorio.

3.2. Sistema Ext2

- **Eficiencia en Acceso:** El uso de inodos permite un acceso mucho más rápido a los datos. Para leer cualquier bloque de un archivo, el sistema solo necesita leer el inodo y desde allí obtiene la dirección directa del bloque de datos (o la del bloque indirecto). Esto es significativamente más rápido que recorrer una lista enlazada.
- **Reducción de la Fragmentación:** Al tener los punteros a bloques de datos agrupados en el inodo (y en bloques indirectos), el sistema operativo puede tomar decisiones más inteligentes sobre la asignación de bloques, intentando mantenerlos contiguos y reduciendo la fragmentación externa.

- **Escalabilidad:** El sistema de punteros indirectos (en esta simulación, solo simple, pero en la realidad incluye dobles y triples) permite que los archivos alcancen tamaños muy grandes de manera eficiente, superando con creces las limitaciones de FAT.
- **Sobrecarga (Overhead):** La desventaja es una mayor complejidad y una ligera sobrecarga de espacio. Cada archivo requiere un inodo, y los archivos grandes requieren bloques adicionales solo para almacenar punteros, no datos.

3.3. Comparación Directa

Característica	FAT	Ext2 (Simplificado)
Método de Asignación	Lista Enlazada	Indexada (Inodos)
Fragmentación	Alta (Externa)	Baja
Acceso Aleatorio	Lento (debe recorrer la cadena)	Rápido (acceso directo desde el inodo)
Tamaño Máx. de Archivo	Limitado	Muy grande (gracias a punteros indirectos)
Metadatos	Mezclados en el directorio	Centralizados en el inodo
Complejidad	Baja	Moderada-Alta

Cuadro 1: Comparación de características entre FAT y Ext2.

El simulador evidencia que para un escenario con muchos archivos pequeños y operaciones simples, la simplicidad de FAT es adecuada. Sin embargo, en cuanto el tamaño de los archivos o el número de operaciones de borrado y reescritura aumenta, el modelo de Ext2 demuestra ser estructuralmente superior, garantizando un rendimiento más consistente y una mejor organización del disco.

4. Conclusiones

El simulador implementado cumple exitosamente su objetivo de modelar y contrastar los sistemas de archivos FAT y Ext2. A través de la implementación de sus estructuras de datos y algoritmos de gestión, se ha podido observar de forma práctica las ventajas y desventajas inherentes a cada arquitectura.

Se concluye que la asignación por lista enlazada de **FAT**, aunque simple, es ineficiente y propensa a la fragmentación, limitando su uso a sistemas más simples o antiguos. Por otro lado, la asignación indexada mediante inodos de **Ext2** representa una solución mucho más robusta, escalable y eficiente, sentando las bases de los sistemas de archivos modernos utilizados en sistemas operativos como Linux.

Este proyecto ha servido como una valiosa herramienta para solidificar la comprensión teórica de la gestión de almacenamiento a bajo nivel.